KARBOWANEC
Account Numbers

Technical Specification — H–I–C Format

Abstract

Karbowanec account numbers provide a compact, human-readable alternative to standard CryptoNote addresses. Rather than inventing a separate naming layer, account numbers are derived directly from the blockchain position of a registration transaction: block height H, transaction index I, and a Luhn mod 36 check character C, yielding identifiers such as 4821033-7-K. The system requires no central registry, no auction mechanism, and no new trust assumptions. It is fully on-chain, deterministic, and compatible with the existing CryptoNote protocol. This document describes the format, registration mechanism, resolution process, security model, and reorganization risk.

1. Motivation

Standard CryptoNote addresses are long base58-encoded strings, typically 97 characters, encoding a public spend key and a public view key. While secure and expressive, they are poorly suited to everyday human use:

- Difficult to communicate verbally or in print
- Error-prone when typed manually
- Impractical to memorize
- Visually indistinguishable from one another at a glance

Account numbers solve the usability problem without replacing addresses. They act as a short, structured reference that resolves to a full address on demand. The analogy is a bank account number: opaque and numeric, but short enough to be read over the phone, written on a card, or included in an invoice.

Design Goal

Account numbers are a usability layer, not a replacement for addresses. Every existing workflow continues to work unchanged. Account numbers are purely additive.

2. Account Number Format

An account number has the form:

```
H-I-C
```

Component	Description
H	Block height at which the registration transaction was mined
I	Transaction index within that block (index 0 is reserved for the coinbase transaction and is not valid for registration)
C	Single check character, Luhn mod 36, over the digit string of H and I concatenated

The check character alphabet is:

```
0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ   (36 characters)
```

2.1 Example Account Numbers

Account Number	Meaning
100000-3-Q	Transaction at index 3 in block 100,000 (3rd non-coinbase transaction)
4821033-7-K	Transaction at index 7 in block 4,821,033 (7th non-coinbase transaction)

2.2 Luhn Mod 36 Check Character

The Luhn mod 36 algorithm is applied to the digit string formed by concatenating the decimal representation of H and I with no separator. For example, for H = 4821033 and I = 7, the input string is “48210337”.

Luhn mod 36 detects all single-digit substitution errors and all adjacent transposition errors — the two most common classes of human transcription mistake. This makes it significantly stronger than a plain modulo checksum, which does not guarantee detection of transpositions.

A typo such as `4821033-7-K` → `4821033-8-?` will never produce the same check character `K`. Validation fails before any network lookup is attempted, preventing silent misdirection of funds.

3. Registration Mechanism

To obtain an account number, a user broadcasts a registration transaction containing their full Karbo address in the transaction extra field. When the transaction is mined into block H at position I, the account number H-I-C is permanently established.

There is no claiming step, no auction, and no namespace to squat. The number is assigned by the network at mining time.

3.1 Transaction Extra Field

A new extra tag is defined:

```
TX_EXTRA_TAG_ACCOUNT_REGISTRATION = 0x04  (next available tag)

Layout: [0x04] [spend_pubkey: 32 bytes] [view_pubkey: 32 bytes]
Total:  65 bytes (fixed, no length prefix required)
```

The full address — public spend key and public view key — is stored explicitly in the extra field. This makes the registration record self-contained: it can be validated and stored without resolving any other data.

3.2 Validation Rules

A registration transaction is valid if and only if all of the following hold:

1. **Tag present exactly once.** A transaction containing two registration tags is malformed and rejected.
2. **No payment ID.** A registration transaction must not contain any payment ID nonce field. The two are mutually exclusive.
3. **Valid curve points.** Both spend_pubkey and view_pubkey must be valid, non-identity Ed25519 points. Transactions with keys that fail curve validation are rejected at ingestion time.
4. **Standard fee.** No elevated fee is required. The standard minimum transaction fee applies.

Only well-formed registrations satisfying all of the above are indexed in the database and eligible for resolution.

4. Resolution

4.1 Canonicity Rule: First Registration Wins

The first registration for a given address — the one at the lowest (H, I) position on the main chain — is the canonical account number for that address. Subsequent registrations for the same address are permitted as valid transactions and will be mined, but they are not inserted into the registration index and will not be resolved by any RPC method.

This gives account numbers a stable, permanent identity. Once established, a canonical account number cannot be displaced by later registrations.

Why First Wins

Security is cleaner: once established, the mapping is immutable. No one can displace an account number by broadcasting a later registration.

Cacheability: a resolved account number never changes meaning. Software can cache results permanently.

Simpler index: insert once, never update under normal conditions.

Spam is pointless: subsequent registrations for an already-indexed address cost fees and have zero effect.

4.2 Forward Lookup (Account Number → Address)

Given an account number `H-I-C`, resolution proceeds as follows:

5. **Validate checksum.** Verify that C is correct for (H, I). Return an error immediately if not — no network query is made.
6. **Query the LMDB index.** Look up the packed key (H ≷ 32 | I) in the account_registrations table.
7. **Return the address.** If found, return the encoded Karbo address. If not found, return “not registered”.

Forward lookup is O(1) — a single database key read.

4.3 Reverse Lookup (Address → Account Number)

Given an address, walk the LMDB index forward from the beginning, returning the first matching entry. Because the index contains only valid first registrations, the first match is by definition the canonical number. Returns at most one result.

5. LMDB Index

A dedicated LMDB table stores all canonical registration records:

```
Table:  account_registrations
Key:    uint64_t  =  (blockHeight << 32) | txIndex
Value:  64 bytes  =  spend_pubkey (32) || view_pubkey (32)
```

The uint64_t key packs height into the high 32 bits and index into the low 32 bits, giving natural chronological order. Only well-formed first registrations are inserted; subsequent registrations for already-indexed addresses are silently skipped.

Operation	Complexity
Forward lookup (account number → address)	O(1) — single key read
Reverse lookup (address → account number)	O(R) — forward scan, R = total registrations
Insert on block add	O(log R)
Delete on block pop (reorg)	O(log R)

The index is populated when a block is added to the main chain and depopulated when a block is popped during a reorganization, keeping it consistent with the main chain at all times.

5.1 Backfill on First Run

On first startup after the index is introduced, the node scans backwards from the current chain tip down to block 1,264,265 — the block containing the first known registration at tx index 1 — and inserts any well-formed first registrations not yet present in the index.

The presence of the entry `(1264265, 1)` in the index is used as the completion marker: if this entry exists, the backfill is considered complete and is not repeated on subsequent startups.

Why the index is required from day one

Without the LMDB index, reverse lookup requires scanning every block in the chain — O(blockchain height) — completely unusable on a public node. The backfill procedure handles the historical chain for existing nodes at upgrade time. Shipping the index from day one avoids a costly later migration for new deployments.

6. Spam Resistance

The system is naturally resistant to spam without requiring an elevated registration fee.

Spamming your own address

Broadcasting many registration transactions for the same address results in only the first being indexed. All subsequent registrations are ignored by the index. There is no benefit to the sender beyond paying fees for no effect.

Spamming someone else's address

An attacker who registers a victim's address first would establish the canonical number — but it points to the victim's address. The attacker cannot intercept funds sent to it. There is no financial upside to this attack, only wasted fees.

Flooding the index

Large-scale registration spam is constrained by the standard fee market. Each spam transaction consumes block space and pays fees; the index only grows by one entry per unique address registered.

Conclusion

Standard minimum transaction fees are sufficient. The first-wins canonicity rule means spamming existing registrations is economically irrational, and spamming new registrations costs fees with no exploitable gain.

7. Reorganization Risk

Account numbers derive their identity from blockchain position. Because Karbowanec uses probabilistic finality — as all proof-of-work chains do — a recently mined block can in rare circumstances be replaced by a competing chain. This section analyzes the consequences for account number integrity.

7.1 The Risk

Suppose a user broadcasts a registration transaction that is mined into block H at index I, yielding account number `H-I-C`. If a reorganization subsequently replaces block H, several outcomes are possible:

- The registration transaction is included again at the same (H, I) position in the new chain — the account number is unaffected.
- The registration transaction is included at a different position (H', I') — a different account number is assigned.
- The registration transaction is not included in the reorganized chain at all — the account number temporarily ceases to exist until the transaction is re-mined.

- A **different** registration transaction for a different address occupies position (H, I) in the new chain — the same account number now resolves to a different address.

Most Dangerous Case

The fourth scenario is the most serious. If an attacker can cause a reorganization and insert a registration for a controlled address at position (H, I), they can make a previously published account number point to their own address. Funds sent to that account number during the reorg window would be misdirected.

7.2 Feasibility of Attack

Executing this attack requires:

8. Observing a target registration transaction in a recently mined block.
9. Mining an alternative chain of sufficient length to displace that block.
10. Including a replacement registration at the same (H, I) position.
11. Broadcasting the alternative chain before the victim shares their account number with third parties.

The computational cost of step 2 scales with the depth of the reorganization required. On Karbowanec, reorganizations deeper than a few blocks are extremely rare under normal network conditions and become exponentially more expensive with each additional block. An attacker would need majority hash power sustained over multiple blocks to reliably execute a deep reorganization.

The attack is also time-constrained: it is only relevant during the window between initial registration and the point at which the account number is widely shared and trusted.

7.3 Mitigation: Confirmation Policy

The standard mitigation is to treat a new account number as unconfirmed until a sufficient number of blocks have been built on top of the registration block, mirroring the confirmation practice already applied to received funds.

Use Case	Min. Confirmations	Rationale
Testing / internal use	1–5 blocks	Reorgs at this depth are rare but possible
Low-value payments	10 blocks	Reasonable protection against short reorgs
High-value or public sharing	20–60 blocks	Deep reorgs are computationally prohibitive
Permanent publication	60+ blocks	Essentially final under normal conditions

7.4 Detection and Recovery

If a reorganization occurs after an account number has been shared, detection is straightforward: the account number either fails to resolve or resolves to an unexpected address. Wallets should:

- **Always display the resolved address** to the user before constructing a payment transaction.
- **Warn on resolution failure** when an account number that previously resolved now returns “not found”.
- **Surface re-registration** as an option if the user’s account number was displaced. Re-broadcasting the registration transaction restores the number once re-mined.

7.5 Summary

Key Points

- Reorg risk is real but bounded by proof-of-work economics.
- The attack requires sustained majority hash power and a narrow time window.
- Waiting for 10–60 confirmations before sharing effectively eliminates practical risk.
- Wallets must always display the resolved address for user verification before sending.
- Recovery is simple: re-broadcast the registration transaction if needed.

8. Privacy Considerations

A registration transaction stores the public spend key and public view key of the registered address on-chain, permanently and publicly.

8.1 What Is Already Public

A standard Karbo address encodes exactly these two keys. Any address shared with another party — for any payment, on any forum, in any message — already exposes them. Registration does not reveal anything that address sharing does not.

8.2 The View Key and Incoming Transactions

The view key allows its holder to identify which transaction outputs belong to the registered address and to see incoming amounts. This is an intentional property of CryptoNote: the view key is designed for auditability. What registration adds is discoverability — the address becomes findable by anyone scanning the registration index. Users who wish to preserve address privacy should not register.

8.3 What Registration Does Not Reveal

Registration does not reveal outgoing transactions, amounts sent, or the identity of transaction counterparties. The ring signature and stealth address properties of CryptoNote are unaffected.

User Guidance

Account numbers are a convenience feature for users who are already sharing their address publicly. Users who prioritize privacy should continue to share addresses directly and selectively, without registering.

9. RPC Interface

Two JSON-RPC methods are provided:

resolve_account_number

```
Request:  { "account_number": "4821033-7-K" }
Response: { "address": "<base58 address>" }
          { "error": "invalid_checksum" | "not_found" }
```

Resolves an account number to its registered address. Checksum is validated locally before any database query is made.

get_account_number

```
Request:  { "address": "<base58 address>" }
Response: { "account_number": "4821033-7-K" }
          { "error": "not_found" }

Returns the canonical (first registered) account number only.
O(R) forward scan of the registration index, stops at first match.
```

Both endpoints are subject to standard rate limiting consistent with other node RPC methods.

10. Wallet Integration

10.1 Sending

When the user enters a payment destination, the wallet accepts either a full Karbo address or an account number. If an account number is detected:

12. **Resolve the number** via resolve_account_number RPC.
13. **Display the resolved address** to the user for confirmation.
14. **Construct the transaction** to the resolved address as normal.

The resolved address must always be shown to the user. They are the final verifier against reorg or lookup errors.

10.2 Receiving

When the user navigates to a receive screen, the wallet checks whether their address has a registered account number via get_account_number. Two cases:

- **Number found:** Display the account number prominently alongside the full address.
- **No number found:** Prompt the user: “You don’t have an account number yet. Register one for easy payments?” If confirmed, broadcast a registration transaction, poll for confirmation, then display the resulting account number.

After confirmation, advise the user to wait for the recommended confirmations before sharing the account number publicly.

11. Summary

Property	Value	Notes
Format	H-I-C	Block height, tx index, check char
Check algorithm	Luhn mod 36	Detects substitutions and transpositions
Check alphabet	0–9 A–Z	36 characters
Registered data	spend_pubkey || view_pubkey	64 bytes in tx extra
Extra tag	0x04 (TBD)	Next available tag
Canonicity rule	First registration wins	Only first is indexed and resolved
Subsequent registrations	Allowed, not indexed	Mined normally, ignored for resolution
Forward lookup	O(1)	Single LMDB key read
Reverse lookup	O(R)	Forward scan over registration index
Fee	Standard minimum	No elevated fee required
Payment ID	Prohibited	Mutually exclusive with registration tag
Recommended confirmations	10–60 blocks	Before sharing publicly
Backfill on upgrade	Scan back to block 1,264,265	Completion marker: entry (1264265,

		1)
Effect on emission	None	Purely additive
Effect on existing addresses	None	Fully backwards compatible